

Anthropic Claude Code

Tool 机制迭代史

从 Function Calling 到 Agent Runtime / Agent OS

研究重点：know-why × know-how × failure-driven evolution

时间范围：2024.11 - 2026.04+ | 基于 Anthropic 官方博客、官方文档与 Claude Code
机制梳理

核心判断 Claude Code 的 Tool 演进不是“不断增加工具数量”，而是在持续重构模型、Context、工具、执行环境与安全边界之间的职责分工。随着能力与自治时长上升，越来越多确定性工作从模型中下沉到 Runtime / Environment。

用途：Agent Runtime 架构设计 / Coding Agent 机制研究 / Tool Design 复盘

执行摘要

如果把 Claude Code 的两年演进压缩成一句话：**把概率性的留给模型，把确定性的逐步移出模型。**

- 早期核心：极简 Agent Loop + Bash/File Tools，让模型真正操作开发环境，而不是只“建议”代码。
- 中期矛盾：工具和外部系统越来越多，Tool Schema、Tool Result 与探索日志开始吞噬 Context。
- 关键转折：Context Engineering、Subagent、Skills、Tool Search、Programmatic Tool Calling 将“按需加载、隔离、压缩、代码编排”变成一等机制。
- 安全演进：从 Human Approval 转向 Permission Policy + Hooks + Sandbox + Classifier，安全边界从“行为监督”升级为“结构性约束”。
- 长任务演进：从单一 Session 转向 compaction、artifact、progress file、evaluator 与跨 Session harness。
- 最终方向：Brain / Hands / Session 解耦，Tool 由“函数”扩展为“统一执行抽象”，Agent 本身也可以成为 Tool。

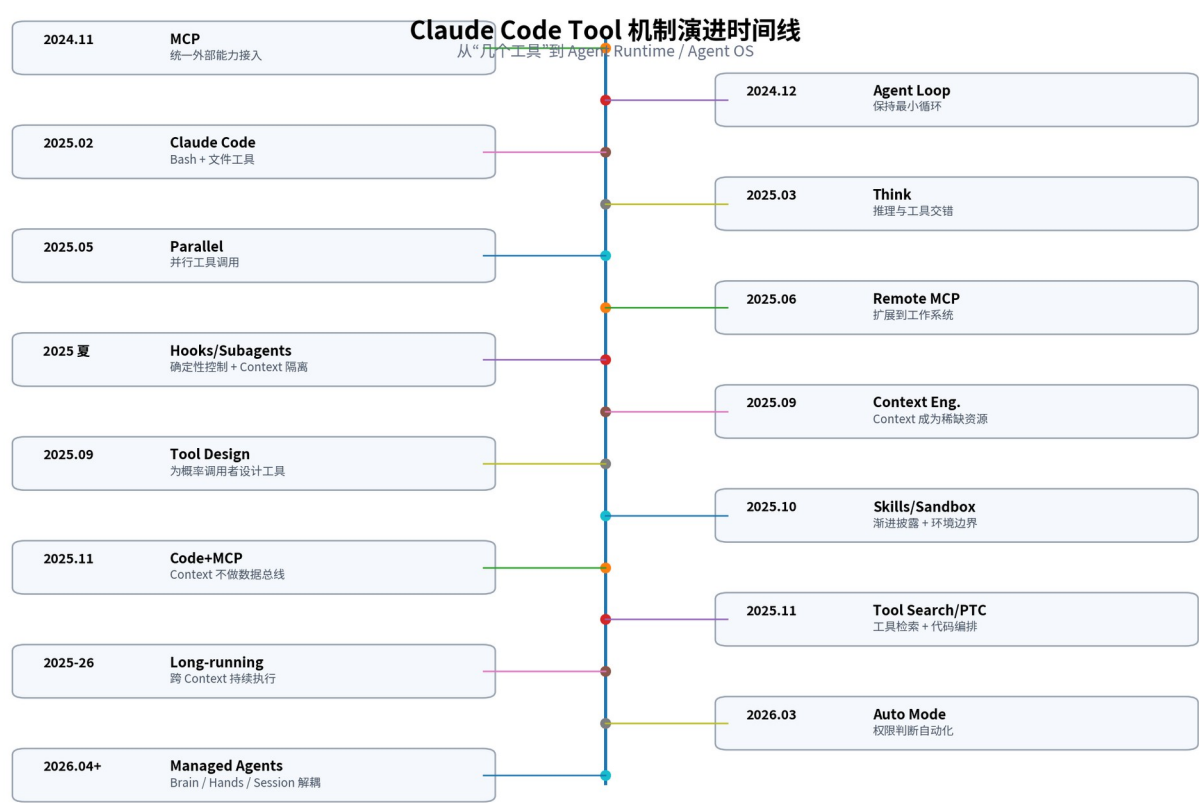


图 1 | Claude Code Tool 机制：从能力接入到分布式 Agent Runtime

最值得记住的 6 个长期原则

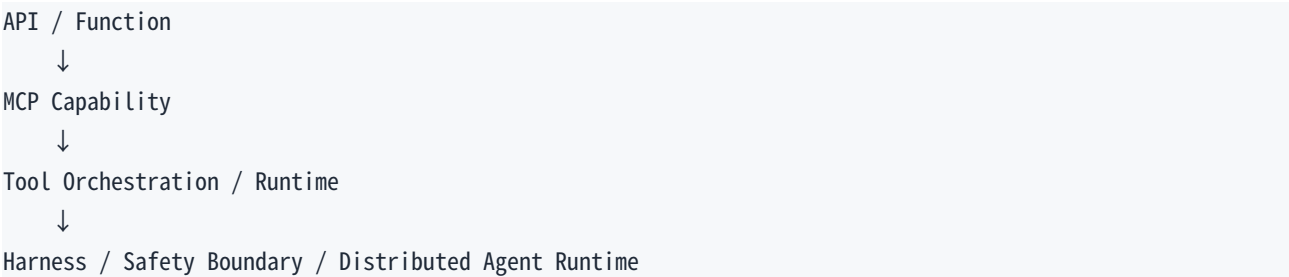
原则	含义
1. Tool 是 Agent affordance，不是 API endpoint	高频、清晰、可约束行为用 Dedicated Tool；长尾组合能力用 Bash/MCP/代码。
2. Context 是稀缺工作内存	不是所有 Tool Schema、Tool Result、探索日志都应该进入主 Context。
3. Progressive Disclosure	工具、技能、知识按需检索/加载，不在启动时把所有可能性塞进系统提示。
4. Deterministic control 下沉	Hooks、Permission、Sandbox、代码循环等确定性逻辑不要只靠 Prompt。
5. Agent itself can be a Tool	Subagent/Workflow 把独立 Context、并行执行与专业化变成可组合能力。
6. Failure-driven Harness	不要设计“终极 Agent 架构”；用 Eval 发现 failure mode，只增加最低必要脚手架，并在模型升级后删除过时机制。

一、全景时间线：15 个阶段与核心矛盾

阶段	时间	机制	核心矛盾	解决方向
0	2024.11	MCP	Integration explosion	用开放协议统一 Tool / Data 接入
0.5	2024.12	Building Effective Agents	Framework complexity	坚持最小 Agent Loop，按 failure mode 增量加机制
1	2025.02	Claude Code Preview	Chat 只能建议，不能执行	Bash + 文件 + Git + 测试，把环境变成可操作状态
2	2025.03	Think / Interleaved reasoning	长 Tool Chain 中间需要重新决策	reason → act → observe → reason
3	2025.04-06	Permission + MCP	能力扩展同时带来风险	Tool Intent 与 Tool Authorization 分离
4	2025.05	Parallel Tool Use	串行 round-trip 高延迟	并行独立 Tool Call
5	2025 夏	Hooks / Subagents	概率性规则 + Context 污染	生命周期 Hook + 独立 Context
6	2025.09	Context Engineering	Tool/Log 挤占工作内存	压缩、隔离、按需加载
7	2025.09-10	Tool Design / Skills	API 不是好 Agent Tool；知识爆炸	Agent Eval + Progressive Disclosure
8	2025.10	Sandbox	Approval fatigue	结构性 filesystem/network containment
9	2025.11	Code Execution with MCP	Schema/result token 爆炸	代码处理 raw data，Context 只留语义结果
10	2025.11	Tool Search + PTC	工具发现与编排成为瓶颈	Tool Retrieval + Programmatic Tool Calling
11	2025-26	Long-running Harness	单 Context 无法覆盖长任务	compaction/reset/artifact/progress/evaluator

阶段	时间	机制	核心矛盾	解决方向
12	2026.03	Auto Mode	人类 93% approve, 审批边界失效	classifier + sandbox 分层监督
13	2026.04+	Managed Agents	运行环境、Session、Harness 强耦合	Brain / Hands / Session 解耦

从 Tool 到 Agent OS：四次抽象层迁移



关键理解 每当模型能力、工具数量、上下文长度或自治时长上升，Anthropic 都会重新问一次：哪些工作必须由模型做？哪些可以移到确定性系统？这比“再增加一个 Tool”更重要。

二、机制迭代详解：每一步为什么发生、怎么实现

Phase 0 | 2024.11 | MCP：先解决 Tool Integration Explosion

问题 / Failure mode	Claude 需要连接 GitHub、Slack、数据库、Drive、Issue Tracker 等大量外部系统；每个模型 × 每个系统的定制集成形成 $N \times M$ 复杂度。
为什么旧方案不够	模型会调用工具，不等于工程上可以稳定、可维护地连接所有工具。继续做一对一 integration 会导致协议、鉴权、资源表示与错误处理全部碎片化。
Anthropic 怎么改	通过 Model Context Protocol 把模型侧抽象为 MCP Client，把能力提供方抽象为 MCP Server，用统一协议暴露 tools/resources/prompts。
可复用工程经验	把“智能决策”与“能力接入协议”分离。Agent Runtime 不应把每个外部服务的集成细节写死在核心循环里。

官方材料线索：Anthropic: Model Context Protocol (2024-11-25).

Phase 0.5 | 2024.12 | Building Effective Agents：保持 Agent Loop 最小

问题 / Failure mode	Agent Framework 容易在没有证据的情况下提前加入 Planner、Router、Critic、Memory 等复杂模块。
为什么旧方案不够	复杂框架会增加不可见状态、调试面和 token/latency 成本，而且很多“假想问题”模型本身已经能解决。
Anthropic 怎么改	从最小闭环开始：LLM(context, tools) → execute tool → append result → continue，只有观测到明确 failure mode 才加脚手架。
可复用工程经验	先跑 trace、再加机制；Harness 应该是对具体 failure 的补丁，而不是架构审美。

```
while stop_reason == "tool_use":
    response = LLM(messages, tools)
    execute tools
    append tool results
```

官方材料线索：Anthropic Engineering: Building Effective Agents (2024-12).

Phase 1 | 2025.02 | Claude Code Preview：Bash 成为 Meta Tool

问题 / Failure mode	普通聊天模型只能告诉开发者“怎么改”，无法观察 repo、运行测试、执行 Git 或调用现有 CLI。
为什么旧方案不够	为 npm、git、pytest、docker、gh、make 等分别定义专用 Tool 会产生 Tool Explosion，并且永远覆盖不了长尾命令。
Anthropic 怎么改	Claude Code 直接进入 terminal 环境，组合 Dedicated File Tools（Read/Edit/Glob/Grep 等）与通用 Bash；Bash 可以调用几乎任意 CLI。
可复用工程经验	“高频 + 可结构化 + 需要约束”的能力做 Dedicated Tool；“长尾 + 强组合”的能力交给 Bash/代码。Tool 粒度应围绕 Agent 可稳定理解的行为单元，而不是 API endpoint。

Read / Edit / Grep / Glob → structured, predictable
Bash → universal, composable, risky

Dedicated tools + one meta-tool > hundreds of endpoint tools

官方材料线索：Anthropic News: Claude 3.7 Sonnet & Claude Code research preview (2025-02-24).

Phase 2 | 2025.03-05 | Think / Interleaved Reasoning：推理不只发生在开头

问题 / Failure mode	长 Tool Chain 中，新的观察结果会改变后续路径；一次性 plan-all-then-execute 容易沿错误假设继续。
为什么旧方案不够	工具调用的价值就是改变信息状态。如果每次 observation 后不能重新判断，Agent 只是机械执行初始计划。
Anthropic 怎么改	引入 think tool，并随后发展为 extended thinking 与 tool use 交错：reason → act → observe → reason。
可复用工程经验	对动态环境优先使用闭环控制，而不是静态长计划。中间 observation 应当能触发策略重算。

官方材料线索：Anthropic Engineering: The “think” tool (2025-03); Claude 4 tool use updates (2025-05).

Phase 3 | 2025.04-06 | Permission：Tool Intent 与 Authorization 分离

问题 / Failure mode	当 Agent 拥有 shell、写文件、网络与外部系统能力后，模型发出的 Tool Call 不应该自动等于授权。
为什么旧方案不够	让模型自己判断“我是否被允许做这件事”是不稳定的；安全边界必须独立于模型意图。
Anthropic 怎么改	Runtime 对 Tool Call 应用 allow/deny/ask 规则，支持 allowedTools、permission rules；后续 Hooks/Sandbox 继续强化此层。
可复用工程经验	把“想做什么”与“能不能做”放在两套系统中。安全策略必须能在模型之外确定性执行。

官方材料线索：Claude Code Best Practices / Permission documentation.

Phase 4 | 2025.05 | Parallel Tool Use：从串行 round-trip 到并行执行

问题 / Failure mode	多个独立搜索、读取、检查操作如果串行执行，每一个都需要 inference + tool latency。
为什么旧方案不够	没有数据依赖的任务被串行化，会把总延迟近似变成各步骤之和；这不是 reasoning 问题，而是调度问题。
Anthropic 怎么改	支持一次生成多个独立 Tool Calls 并并行执行，再将结果合并返回模型。
可复用工程经验	把 Agent trace 当作 DAG，而不是链表。没有依赖关系的节点默认并发；不要让 LLM 轮询式串行做可并行工作。

串行： $\Sigma(\text{Inference}_i + \text{Tool}_i)$
并行： $\text{Inference} + \max(\text{Tool}_1, \text{Tool}_2, \dots \text{Tool}_n)$

官方材料线索：Claude 4 announcement / parallel tool use capabilities.

Phase 5A | 2025 夏 | Hooks：把 deterministic control 从 Prompt 拿出来

问题 / Failure mode	“每次改 TS 后必须 lint” “某类命令禁止执行” 之类规则如果只写在 CLAUDE.md / prompt，模型可能忘记或误解。
为什么旧方案不够	Prompt 是概率约束，不能替代生命周期 enforcement。规则越重要，越不应该依赖模型记忆。
Anthropic 怎么改	在 PreToolUse、PostToolUse、PermissionRequest、Stop、SessionStart 等生命周期点执行 Hook；可 deny、modify input、注入 context 或运行校验。
可复用工程经验	可以 deterministic enforcement 的东西不要只写 Prompt。把 policy 从“文字建议” 升级为 Runtime 生命周期机制。

官方材料线索：Claude Code Hooks documentation.

Phase 5B | 2025 夏 | Subagents：真正价值是 Context Isolation

问题 / Failure mode	探索型任务会产生大量 grep/read/git/log/tool output，全部写回 Main Context 会快速污染工作内存。
为什么旧方案不够	主 Agent 真正需要的是结论、证据和下一步，而不是所有中间过程。并行只是附带收益，隔离才是核心。
Anthropic 怎么改	Agent tool 启动独立 context window；subagent 自己读文件、搜索、调用工具，父 Agent 只接收最终摘要/结果。
可复用工程经验	Subagent 可以理解为 Context Garbage Collection：把高噪声工作放进临时 Context，只把压缩后的高价值信息返回主线。

官方材料线索：Claude Code Tools Reference / Subagents documentation.

Phase 6 | 2025.09 | Context Engineering：Tool 问题升级为 Context 问题

问题 / Failure mode	随着 Tool 数量、Tool Result、历史 trace 增长，Agent 的 Context Window 被 schema、日志和冗余结果挤占，出现 context rot。
为什么旧方案不够	Context Window 不是无限知识库，而是每次推理的有限 working memory；把所有历史都保留会降低模型注意力质量。
Anthropic 怎么改	采用 compaction、summary、subagent 隔离、按需加载工具/技能，只保留架构决策、未解决问题、近期关键文件与高价值结果。
可复用工程经验	优化对象不再只是 prompt，而是“每一次模型调用时完整的信息状态”。Context budget 应像内存预算一样管理。

官方材料线索：Anthropic Engineering: Effective Context Engineering for AI Agents (2025-09).

Phase 7A | 2025.09 | Writing Effective Tools：Tool 是给概率调用者的接口

问题 / Failure mode	传统 API 只需参数合法、结果正确；Agent Tool 还必须让模型知道何时调用、如何区分相似工具、如何填参数、如何读结果。
为什么旧方案不够	更多工具并不自动提升 Agent。相似命名、模糊 schema、巨大 result 都会增加 routing / parameter / context error。
Anthropic 怎么改	明确能力边界、使用 namespace、限制输出字段、优化 description、提供 examples，并为 Tool 建立 Agent Eval，观察 trace 反向改 schema。
可复用工程经验	Tool Description 本身就是 Prompt；Tool Schema 是模型 affordance。为 Agent 设计工具，需要做可观测、可评测、可迭代的接口工程。

官方材料线索：Anthropic Engineering: Writing effective tools for agents — with agents (2025-09).

Phase 7B | 2025.10 | Skills：解决 Instruction / Knowledge Explosion

问题 / Failure mode	企业 Agent 需要大量流程知识、组织规范、模板、脚本和领域经验；全部塞进 System Prompt 会长期占 Context。
为什么旧方案不够	知识“可能有用”不代表“此刻需要”。启动时加载全部技能会与 Tool Schema Explosion 一样浪费工作内存。
Anthropic 怎么改	Skill 目录包含 SKILL.md、脚本、模板和资源；启动只暴露名称/描述，相关时再加载完整 skill，必要时再读取资源。
可复用工程经验	把领域能力做成 Progressive Disclosure：metadata → instructions → resource/script。知识与工具采用相同的 JIT loading 思路。

官方材料线索：Anthropic Engineering: Equipping agents for the real world with Agent Skills (2025-10).

Phase 8 | 2025.10 | Sandbox：从 Approval Safety 到 Structural Safety

问题 / Failure mode	Bash/写文件/网络操作频繁弹 permission prompt，用户逐渐形成机械 approve 的 approval fatigue。
为什么旧方案不够	“每次都问人”理论安全、实际不一定安全；当确认率过高，人类审核不再是可靠 boundary。
Anthropic 怎么改	通过 OS 级 filesystem + network sandbox，把常规操作限制在允许边界内；边界内自由执行，越界才升级审批。
可复用工程经验	不要只问 Agent/用户“可不可以做”，尽可能让环境直接决定“做不到”。安全从行为监督升级为 capability bounding。

官方材料线索：Anthropic Engineering: Claude Code Sandboxing / Beyond permission prompts (2025-10).

Phase 9 | 2025.11 | Code Execution with MCP：Context 不应该做数据总线

问题 / Failure mode	数百个 MCP Tool 的 schema 占大量 token；每次工具返回 raw JSON 又全部写入 Context，模型被迫承担数据搬运与处理。
为什么旧方案不够	LLM 擅长语义判断，不擅长把 Context 当程序内存做 loops/filter/aggregation；频繁 round-trip 同时增加 token 和 latency。
Anthropic 怎么改	让模型生成 Python/JS，在 execution runtime 中循环、并行、过滤、聚合 MCP Tool Result，只把最终高价值摘要返回 Context。
可复用工程经验	把 raw data 留在执行环境，让 Context 只承载决策相关语义。Context Window 不应成为 data processing bus。

Claude → writes code
Python/JS → call tools → filter → aggregate → handle errors
Claude ← receives compact semantic result

官方材料线索：Anthropic Engineering: Code execution with MCP (2025-11).

Phase 10A | 2025.11 | Tool Search：从“全量注入”到 Tool Retrieval

问题 / Failure mode	工具数量达到几十/几百后，把所有 schema 放进 prompt 不仅贵，还会降低工具选择准确率。
为什么旧方案不够	Tool 也是一种“可能相关的信息”。全量注入违背 Context Engineering；真正需要的是按任务检索能力。
Anthropic 怎么改	通过 defer loading / Tool Search，只暴露少量通用工具和搜索入口，根据任务检索 3-5 个相关 Tool 再加载 schema。
可复用工程经验	Tool Search 本质是 RAG for Tools。工具集合越大，越应把 discovery 与 invocation 解耦。

官方材料线索：Anthropic Engineering: Advanced Tool Use (2025-11).

Phase 10B | 2025.11 | Programmatic Tool Calling：控制流从自然语言迁移到代码

问题 / Failure mode	五步工具流程若每一步都回到模型，形成多次 inference round-trip；而循环、排序、并行、重试本质是确定性程序逻辑。
为什么旧方案不够	让 LLM 用自然语言维持控制流既贵又不可靠；Python/JS 对 if/loop/filter/async/error handling 更适合。
Anthropic 怎么改	模型负责生成程序与语义策略，Runtime 负责执行多 Tool orchestration，中间结果不必全部进入模型。
可复用工程经验	LLM 决定 WHAT/WHY，代码执行 HOW。能写成确定性控制流的部分优先下沉到 code runtime。

官方材料线索：Anthropic Engineering: Advanced Tool Use — Programmatic Tool Calling (2025-11).

Phase 10C | 2025.11 | Tool Use Examples : Schema 只描述 Syntax , 不描述 Semantics

问题 / Failure mode	JSON Schema 能说明字段类型，却不能完整表达“什么情况下 priority=critical、什么时候需要 escalation”。
为什么旧方案不够	模型调用失败往往不是语法错误，而是语义边界不清；单纯增加字段描述不一定足够。
Anthropic 怎么改	在 Tool definition 中提供 input examples / few-shot usage，让模型看到典型参数组合与真实决策边界。
可复用工程经验	Schema 解决合法性，Examples 解决 affordance。复杂 Tool 应同时做结构约束和示例驱动。

官方材料线索：Anthropic Engineering: Advanced Tool Use — Tool use examples (2025-11).

Phase 11 | 2025.11-2026 | Long-running Harness：让 Tool Loop 跨越 Context Window

问题 / Failure mode	复杂软件任务持续数小时甚至更久，单一 Context 无法承载完整历史；接近上限时还可能提前收尾或失去细节。
为什么旧方案不够	仅靠一次 compaction 仍可能丢失执行状态、任务进度和验收标准。长任务需要环境级持久化，而不是只依赖语言模型记忆。
Anthropic 怎么改	使用 initializer/coding agent、feature list、progress file、git/test artifacts、context compaction/reset；后续加入 Planner/Generator/Evaluator 与 sprint contract。
可复用工程经验	Environment as Memory：让“下一班 Agent”通过 repo、git、测试、任务文件和 artifacts 恢复状态。Harness 是跨 Session 的状态机。

官方材料线索：Anthropic Engineering: Effective harnesses for long-running agents; Harness design for long-running apps.

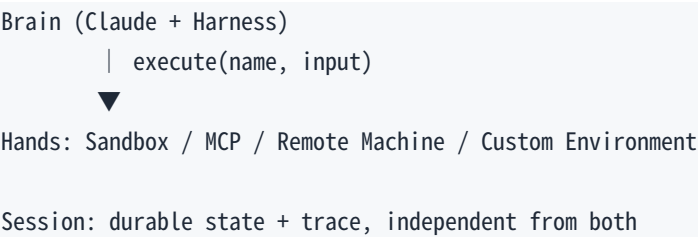
Phase 12 | 2026.03 | Auto Mode：Human-in-the-loop 开始被自动监督取代

问题 / Failure mode	实际使用中大量 permission prompt 被用户直接批准，审批动作本身逐渐变成低信息量点击。
为什么旧方案不够	当人工批准率极高时，人类并没有真正审计每个动作；继续依赖 prompt approval 会产生安全错觉。
Anthropic 怎么改	在 Tool Result 进入上下文前做 prompt-injection probe，在 Tool Call 执行前用独立 classifier 检查 transcript/action；仍与 sandbox 组合使用。
可复用工程经验	安全应分层：模型判断、分类器监督、Runtime policy、OS containment。概率分类器不能替代结构性隔离。

官方材料线索：Anthropic Engineering: Claude Code Auto Mode; How we contain Claude (2026).

Phase 13 | 2026.04+ | Managed Agents : Brain / Hands / Session 解耦

问题 / Failure mode	如果 Claude、Harness、Session、Filesystem、Credentials、Tools 全部绑定在同一个 container，容器崩溃、扩缩容、跨网络/VPC 执行都会很困难。
为什么旧方案不够	执行环境是短生命周期的，但会话状态和推理状态应该可持久；Hands 也可能是 sandbox、MCP、远端机器或客户环境。
Anthropic 怎么改	将 Brain（Claude+Loop）、Session（durable log/state）与 Hands（执行后端）拆分，通过统一 execute(name,input) contract 调用不同能力。
可复用工程经验	Tool 最终变成“统一执行抽象”。Agent Runtime 应将推理、持久状态和执行环境解耦，支持弹性、隔离与多后端。



官方材料线索：Anthropic Engineering: Scaling Managed Agents — Decoupling the brain from the hands (2026-04+).

三、把所有迭代串成一条架构因果链

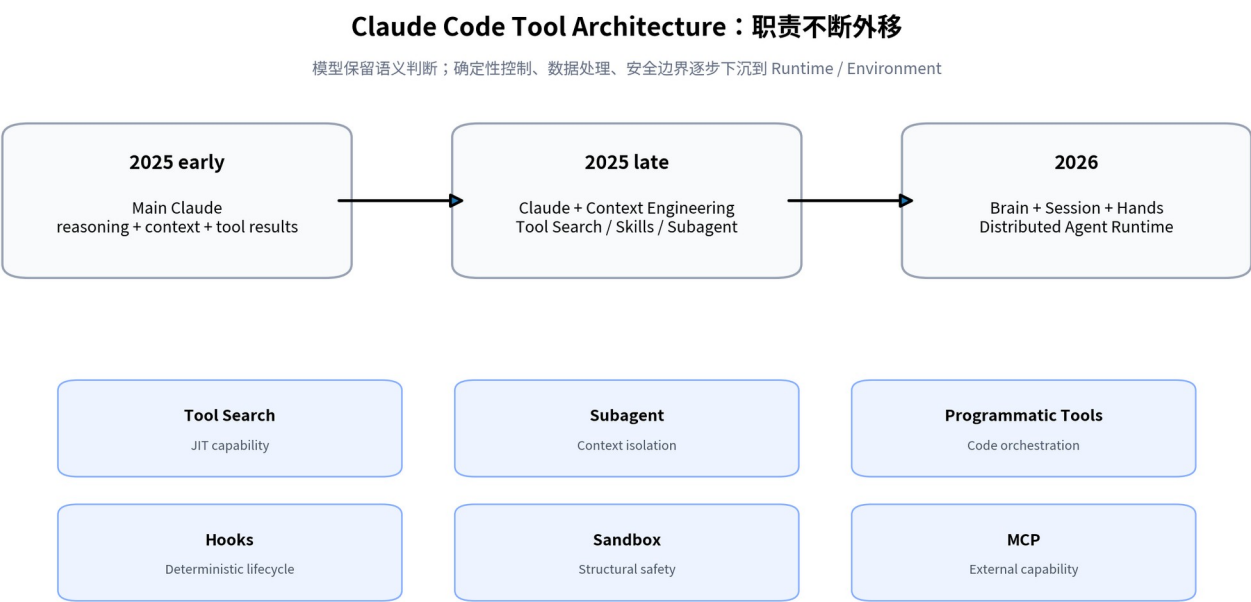


图 2 | Tool 的职责不断从模型 Context 外移到 Runtime 与 Environment

因果链 1：能力增加 → Context 爆炸 MCP 让能力数量迅速上升；能力越多，schema 越多；调用越多，result 越多。于是“接更多工具”最终变成“怎么让不相关工具和中间数据不进入 Context”。

因果链 2：自治增加 → 安全边界必须结构化 Agent 从读代码到能写文件、跑 shell、访问网络后，Prompt 和人工确认都不足以作为唯一安全机制；因此出现 Hooks、Sandbox、Classifier 与多层 containment。

因果链 3：任务变长 → Session 必须独立于 Context Compaction 只是降低 token 占用，真正的长任务需要通过 git、progress file、test artifacts、durable session 把状态放到模型外部。

因果链 4：模型变强 → Harness 应当变薄 很多脚手架本质上编码“当前模型做不到什么”。模型升级后，旧机制可能成为 overhead。正确动作是重新跑 Eval，删除不再必要的 scaffold。

架构层级的最终分工

层	职责	为什么放这里
LLM / Brain	意图理解、语义推理、计划、代码生成、模糊决策	概率性、需要语义能力
Tool Contract	能力描述、schema、examples、namespace	让模型稳定选择/调用
Runtime	Tool Search、并行调度、Programmatic Calling、Hooks	确定性控制流与生命周期
Security Layer	permission policy、classifier、prompt-injection probe	独立于主 Agent 的监督
Environment / Hands	sandbox、filesystem、network、remote machine、MCP	真正执行动作并提供结构性边界
Session / Memory	trace、progress、artifacts、git、task state	跨 Context / 跨进程持续状态

四、核心 know-how：自己做 Agent Runtime 时怎么抄

“把概率性的留给模型，把确定性的移出模型”

Claude Code Tool 机制演进背后的统一设计原则

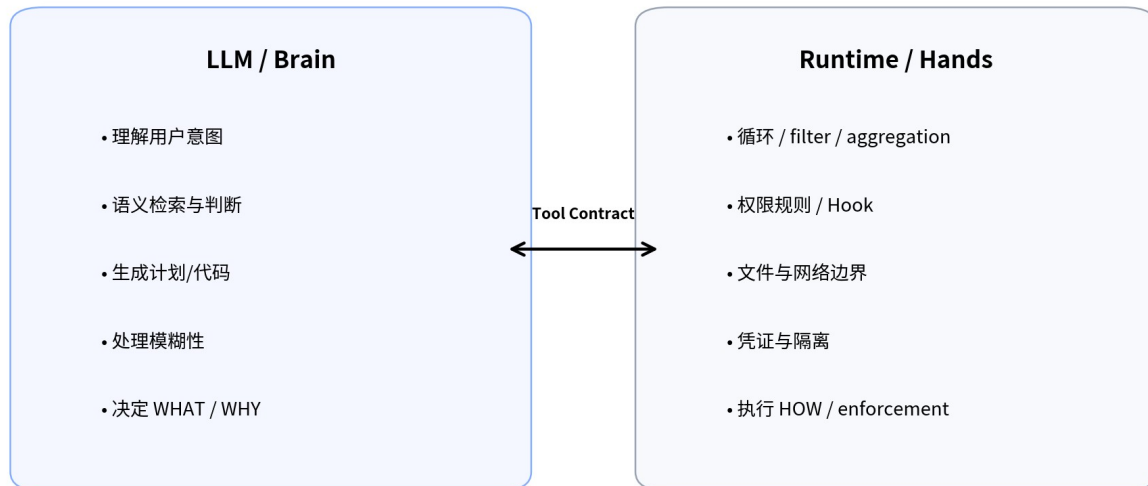


图 3 | 统一原则：概率性决策留给模型，确定性控制下沉到系统

Step 1 | 先做最小闭环

LLM + messages + Read/Edit/Bash + tool_result loop。先获得可观察 trace，不要提前堆 Planner/Memory/Router。

Step 2 | 外部系统多了，上 MCP

统一 capability integration；核心 Agent Loop 不感知每个服务的定制协议。

Step 3 | 工具太多，上 Tool Search

只加载当前任务相关 schema，避免 Tool Definition 占满 Context。

Step 4 | Tool Result 太大，上 Code Execution

让 Python/JS 处理循环、聚合、过滤、并行，只把 compact result 返回模型。

Step 5 | 主 Context 被探索污染，上 Subagent

独立 context 完成搜索/验证/调查，父 Agent 只接收结果摘要。

Step 6 | 领域知识越来越多，上 Skills

把组织知识、模板、脚本做成 progressive disclosure。

Step 7 | 规则必须 100% 执行，上 Hooks/Policy

不要继续往 Prompt 塞“必须/禁止”；把规则做成 runtime enforcement。

Step 8 | 权限确认过多，上 Sandbox

把允许范围结构化；边界内自由执行，越界才升级。

Step 9 | 任务超过 Context，上 Durable State

git、progress、task list、artifact、test results 作为环境记忆；必要时 compaction/reset。

Step 10 | 长任务质量不稳，才加 Planner/Evaluator

把“定义 done”“生成”“验收”拆开，并用独立 evaluator 对抗自我评估偏差。

Step 11 | 部署复杂后，拆 Brain / Hands / Session

统一 execute contract；允许 sandbox、MCP、remote machine 等执行后端独立扩缩容。

真正应该复制的不是“Claude Code 当前有多少 Tool”而是它的迭代方式：先让简单 loop 工作 → 收集 trace/eval → 找到真实 failure mode → 增加最小必要机制 → 模型升级后重新验证并删除过时 scaffold。

Tool Quality：比“工具数量”更有解释力的评价框架

$$\text{Agent Performance} \approx \text{Model} \times \text{Tool Affordance} \times \text{Context Quality} \times \text{Harness} \times \text{Environment}$$
$$\text{Tool Quality} = \text{Selection Accuracy} \times \text{Parameter Accuracy} \times \text{Result Relevance} \times \text{Token Efficiency} \times \text{Safety}$$

- Selection Accuracy：模型是否知道什么时候该用这个 Tool、能否和相似 Tool 区分。
- Parameter Accuracy：Schema、description、examples 是否足以让模型稳定填对参数。
- Result Relevance：返回是否只包含下一步决策所需信息，而非巨大 raw payload。
- Token Efficiency：Tool schema/result 是否采用 defer loading、压缩、代码聚合。
- Safety：授权、生命周期、环境边界是否独立于模型意图。

Agent Tool Design Checklist

- ☐ 这个能力是否真的需要独立 Tool？还是 Bash / code / existing CLI 已足够？
- ☐ Tool 名称、namespace、description 是否能和相近能力清晰区分？
- ☐ Schema 是否约束到“模型容易填对”，而不是只满足 API 合法性？
- ☐ 是否需要真实 input examples 来表达语义边界？
- ☐ Tool Result 是否可以过滤/摘要，避免 raw data 直接写入 Context？
- ☐ 独立调用之间是否可以并行？复杂控制流是否应下沉到 Python/JS？
- ☐ 该 Tool 是否应 defer loading / 通过 Tool Search 按需发现？
- ☐ 高噪声工作是否应放到 Subagent，避免污染主 Context？
- ☐ 必须执行的规则是否已经放到 Hook/Policy，而不是只写 Prompt？
- ☐ 危险能力是否被 Sandbox/网络/文件系统边界限制？
- ☐ 是否为 Tool 建立了基于真实 Agent trace 的 Eval？
- ☐ 模型升级后，这个 Tool/Harness 机制是否仍然必要？

五、Anthropic 官方材料：建议阅读顺序

顺序	材料	重点	官方链接
1	Building Effective Agents	Agent Loop 与 “少框架、按 failure mode 增量” 的底层哲学	https://www.anthropic.com/engineering/building-effective-agents
2	Claude Code: Best Practices	Claude Code 早期 Bash/File/MCP/Permission 使用模型	https://www.anthropic.com/engineering/claude-code-best-practices
3	The “think” tool	reasoning 与 tool observation 之间的交错	https://www.anthropic.com/engineering/claude-think-tool
4	Writing effective tools for agents — with agents	Tool affordance、description、eval、agent-facing API 设计	https://www.anthropic.com/engineering/writing-tools-for-agents
5	Effective context engineering for AI agents	Tool use 为什么最终变成 Context 管理问题	https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents
6	Equipping agents for the real world with Agent Skills	Progressive disclosure 与能力模块化	https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills
7	Claude Code sandboxing	从 permission prompts 到结构性 containment	https://www.anthropic.com/engineering/claude-code-sandboxing
8	Code execution with MCP	为什么 Context 不该做 Tool Result 数据总线	https://www.anthropic.com/engineering/code-execution-with-mcp
9	Advanced Tool Use	Tool Search、Programmatic Tool Calling、Tool Examples	https://www.anthropic.com/engineering/advanced-tool-use
10	Effective harnesses for long-running agents	长任务、跨 Context 状态与 artifact	https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents
11	Harness design for long-running apps	Planner/Generator/Evaluator，以及 Harness 随模型能力变化	https://www.anthropic.com/engineering/harness-design-long-running-apps

顺序	材料	重点	官方链接
12	Claude Code Auto Mode	Tool Authorization 的 classifier 化	https://www.anthropic.com/engineering/claude-code-auto-mode
13	Scaling Managed Agents	Brain / Hands / Session 解耦	https://www.anthropic.com/engineering/managed-agents

六、最终结论：Claude Code Tool 机制真正演进了什么？

- 1 | Tool 从 Function 变成 Capability** 最初是函数调用，后来 Tool 可以是 CLI、MCP 服务、Subagent、Workflow、远端执行环境。
- 2 | Context 从“历史容器”变成“稀缺工作内存”** Tool schema/result、探索过程、领域知识都开始采用检索、隔离、压缩、按需加载。
- 3 | Tool Calling 从自然语言编排变成 Runtime Orchestration** 并行、循环、过滤、聚合、错误处理逐步交给 Python/JS/runtime，而不是反复 LLM round-trip。
- 4 | Safety 从 Human Approval 变成 Capability Bounding** Permission、Hooks、Classifier、Sandbox、Network/Filesystem containment 形成多层安全边界。
- 5 | Agent 从单 Session 变成持久运行系统** Git、Artifacts、Progress、Evaluator、Durable Session 让任务跨 Context、跨进程继续。
- 6 | 最终架构接近 Agent OS** Brain 负责语义与决策，Hands 负责执行，Session 负责持久状态；Tool 是连接三者的统一 contract。

最核心的 know-how：不要追求“工具最多、框架最复杂”的 Agent。追踪真实 failure mode，把不需要概率推理的工作持续下沉到 deterministic runtime；同时把 Context 只留给当前决策真正需要的高价值信息。